

7. Bit Manipulation

Contents

Binary Trie (s)	1
Xor Basis	3
N-SAT	4
Max Xor Subset In Range	7
and (&) in range [l, r]	7
Dynamic Bitset	8
Bit Twiddle Permute	10

Binary Trie (s)

```
1 // MXN should generally be (Max Queries * MXB)
2 const int MXN = 500500 * 30, S = 2, MXB = 30;
3 int nxt[MXN][S], cnt[MXN], nodes = 1;
4 // clear the arrays you add here in clear() function
5 void clear() {
6     for (int i = 0; i < nodes; i++) {
7         nxt[i][0] = nxt[i][1] = 0;
8         cnt[i] = 0;
9     }
10    nodes = 1;
11 }
12 void trieinsert(long long val) {
13     int u = 0;
14     for (int i = MXB - 1; i >= 0; i--) {
15         int bit = (val >> i) & 1;
16         if (!nxt[u][bit]) nxt[u][bit] = nodes++;
17         u = nxt[u][bit];
18         cnt[u]++;
19     }
20 }
21 int triecount(long long val) {
22     int u = 0;
23     for (int i = MXB - 1; i >= 0; i--) {
24         int bit = (val >> i) & 1;
25         if (!nxt[u][bit]) return 0;
26         u = nxt[u][bit];
27     }
28     return cnt[u];
29 }
```

```

30 void trieerase(long long val) {
31     if (!triecount(val)) return;
32     int u = 0;
33     for (int i = MXB - 1; i >= 0; i--) {
34         int bit = (val >> i) & 1;
35         u = nxt[u][bit];
36         cnt[u]--;
37     }
38 }
39 long long get_max_xor(long long val) {
40     if (nodes == 1 || cnt[nxt[0][0]] + cnt[nxt[0][1]] == 0) return -1;
41     int u = 0;
42     long long ans = 0;
43     for (int i = MXB - 1; i >= 0; i--) {
44         int bit = (val >> i) & 1;
45         int opp = bit ^ 1;
46         if (nxt[u][opp] && cnt[nxt[u][opp]] > 0) {
47             ans |= (1LL << i);
48             u = nxt[u][opp];
49         } else {
50             u = nxt[u][bit];
51         }
52     }
53     return ans;
54 }
55 long long get_min_xor(long long val) {
56     if (nodes == 1 || cnt[nxt[0][0]] + cnt[nxt[0][1]] == 0) return -1;
57     int u = 0;
58     long long ans = 0;
59     for (int i = MXB - 1; i >= 0; i--) {
60         int bit = (val >> i) & 1;
61         if (nxt[u][bit] && cnt[nxt[u][bit]] > 0) {
62             u = nxt[u][bit];
63         } else {
64             ans |= (1LL << i);
65             u = nxt[u][bit ^ 1];
66         }
67     }
68     return ans;
69 }
70 // Returns the count of numbers 'x' in the trie such that (x ^ val) < k
71 int query(long long val, long long k) {
72     int u = 0, ans = 0;
73     for (int i = MXB - 1; i >= 0; i--) {
74         if (!u) break;
75         int v_bit = (val >> i) & 1;
76         int k_bit = (k >> i) & 1;
77
78         if (k_bit == 1) {
79             // If k's bit is 1, taking v_bit makes the XOR bit 0, which is strictly less than
80             // We add all elements in that subtree to our answer.
81             if (nxt[u][v_bit]) ans += cnt[nxt[u][v_bit]];
82             // Then we traverse down the opposite branch to evaluate lower bits.
83             u = nxt[u][v_bit ^ 1];
84         } else {
85             // If k's bit is 0, we MUST take v_bit to keep the XOR bit 0.
86             u = nxt[u][v_bit];

```

```

87     }
88 }
89 return ans;
90 }
91 // clear() insert(), remove() query()

```

Xor Basis

```

1  template<const int Log = 30, typename T = ll>
2  struct basis {
3      int sz = 0;
4      array<T, Log> a{};
5      void add(T x) {
6          if(sz == Log) return;
7          int i;
8          while(x) {
9              if(!a[i = __lg(x)]) return sz++, void(a[i] = x);
10             x ^= a[i];
11         }
12     }
13     T reduce(T x) {
14         if(sz == Log) return 0;
15         T res = 0;
16         int i;
17         while(x) {
18             if(a[i = __lg(x)]) x ^= a[i];
19             else res |= T(1) << i, x ^= T(1) << i;
20         }
21         return res;
22     }
23     bool find(T x) {
24         if(sz == Log) return true;
25         int i;
26         while(x) {
27             if(a[i = __lg(x)]) x ^= a[i];
28             else return false;
29         }
30         return true;
31     }
32     void clear() {
33         if(sz) a.fill(0), sz = 0;
34     }
35     T getMax(T r = 0) {
36         for(int i = Log - 1; i >= 0; i--) r = max(r ^ a[i], r);
37         return r;
38     }
39
40     T find_k(size_t k, T base_val = 0) {
41         assert(k < 1ULL << sz);
42         T curr = base_val;
43         for(int i = Log - 1, b = sz - 1; i >= 0; i--) {
44             if(a[i]) {
45                 if((k >> b & 1) ^ (curr >> i & 1)) curr ^= a[i];
46                 b--;
47             }

```

```

48     }
49     return curr;
50 }
51
52 T getMaxBounded(T limit, T r = 0) {
53     T best = -1;
54     for (int i = Log - 1; i >= 0; i--) {
55         if (a[i]) {
56             T r1 = r, r2 = r ^ a[i];
57             if ((r1 >> i) & 1) swap(r1, r2);
58
59             if ((limit >> i) & 1) {
60                 T temp = r1;
61                 for (int j = i - 1; j >= 0; j--)
62                     temp = max(temp ^ a[j], temp);
63                 best = max(best, temp);
64                 r = r2;
65             } else
66                 r = r1;
67         } else {
68             int bit_r = (r >> i) & 1;
69             int bit_l = (limit >> i) & 1;
70
71             if (bit_l == 1 && bit_r == 0) {
72                 T temp = r;
73                 for (int j = i - 1; j >= 0; j--)
74                     temp = max(temp ^ a[j], temp);
75                 best = max(best, temp);
76                 r = -1;
77                 break;
78             }
79             if (bit_l == 0 && bit_r == 1) {
80                 r = -1;
81                 break;
82             }
83         }
84     }
85     if (r != -1) best = max(best, r);
86
87     return best;
88 }
89
90 friend basis operator+(basis const &l, basis const &r) {
91     if(l.sz == Log) return l;
92     if(r.sz == Log) return r;
93     auto res = l;
94     for(int i = 0; i < Log; i++) if(r.a[i]) res.add(r.a[i]);
95     return res;
96 }
97 };

```

N-SAT

```

1 struct nsat {
2     int n;
3     vector<vector<int>> lit, g;

```

```

4     vector<int> occ, pos, neg, val, stk, lvl;
5
6     nsat(int n) : n(n), g(2 * n), occ(2 * n), val(n, -1) {}
7
8     // Pass a list of pairs: {variable_index, is_true}
9     // Example: { {0, 1}, {1, 0} } -> (x_0 OR NOT x_1)
10    void add_clause(initializer_list<pair<int, int>> vars) {
11        vector<int> c;
12        for (auto [u, is_true] : vars) {
13            int L = (u << 1) | (!is_true);
14            c.push_back(L);
15            g[L].push_back(lit.size());
16            occ[L]++;
17        }
18        lit.push_back(c);
19    }
20
21    void apply(int L) {
22        val[L >> 1] = !(L & 1);
23        stk.push_back(L);
24        for (int c : g[L]) {
25            if (pos[c]++ == 0)
26                for (int u : lit[c]) occ[u]--;
27        }
28        for (int c : g[L ^ 1]) neg[c]++;
29    }
30
31    void undo() {
32        int L = stk.back();
33        stk.pop_back();
34        val[L >> 1] = -1;
35        for (int c : g[L]) {
36            if (--pos[c] == 0)
37                for (int u : lit[c]) occ[u]++;
38        }
39        for (int c : g[L ^ 1]) neg[c]--;
40    }
41
42    bool deduce(int &q_head) {
43        while (q_head < stk.size()) {
44            int L = stk[q_head++];
45            for (int c : g[L ^ 1]) {
46                if (pos[c] > 0) continue; // Clause already satisfied
47                if (neg[c] == lit[c].size()) return false; // Conflict found
48
49                if (neg[c] + 1 == lit[c].size()) { // Unit clause triggered
50                    for (int u : lit[c]) {
51                        if (val[u >> 1] == -1) {
52                            apply(u);
53                            break;
54                        }
55                    }
56                }
57            }
58        }
59        return true;

```

```

60     }
61
62     bool ok() {
63         pos.assign(lit.size(), 0);
64         neg.assign(lit.size(), 0);
65         val.assign(n, -1);
66         stk.clear();
67         lvl.clear();
68         int q_head = 0;
69
70         // Force evaluate base unit clauses
71         for (int c = 0; c < lit.size(); ++c) {
72             if (lit[c].empty()) return false;
73             if (lit[c].size() == 1) {
74                 int L = lit[c][0];
75                 if (val[L >> 1] == -1) apply(L);
76                 else if (val[L >> 1] != !(L & 1)) return false;
77             }
78         }
79
80         // Sort variables by frequency for fast O(1) branching
81         vector<int> order(n);
82         iota(order.begin(), order.end(), 0);
83         sort(order.begin(), order.end(), [&](int a, int b) {
84             return occ[a << 1] + occ[a << 1 | 1] > occ[b << 1] + occ[b << 1 | 1];
85         });
86
87         while (true) {
88             if (deduce(q_head)) {
89                 // Find next unassigned variable to branch on
90                 int s = -1;
91                 for (int v : order) {
92                     if (val[v] == -1) { s = v; break; }
93                 }
94
95                 // If all variables are assigned, we found a satisfying assignment
96                 if (s == -1) return true;
97
98                 lvl.push_back(stk.size());
99                 apply(s << 1); // Guess that variable 's' is True
100             } else {
101                 // Conflict! Backtrack to last decision
102                 if (lvl.empty()) return false;
103
104                 int backtrack_idx = lvl.back();
105                 lvl.pop_back();
106
107                 int last_guess = stk[backtrack_idx];
108                 while (stk.size() > backtrack_idx) undo();
109
110                 q_head = stk.size();
111                 apply(last_guess ^ 1); // Flip the guess
112             }
113         }
114     }
115 };

```

Max Xor Subset In Range

```
1 // Given queries L,R find max XOR subset in range
2 const int N = 5e5 + 5;
3 vector<pair<int,int>>qqq[N];
4 int ans[N],basis[22],arr[N],last[22];
5
6 void add(int ind,int x){
7     for (int i = 21; i >= 0; --i) {
8         if((x >> i & 1) == 0)continue;
9         if(ind > last[i]){
10             swap(x,basis[i]);
11             swap(ind,last[i]);
12         }
13         x ^= basis[i];
14     }
15 }
16 int query(int ind){
17     int ret = 0;
18     for (int i = 21; i >= 0; --i) {
19         if(last[i] >= ind){
20             ret = max(ret,ret ^ basis[i]);
21         }
22     }
23     return ret;
24 }
25
26 void solve() {
27     int n;cin >> n;
28     memset(last,-1,sizeof last);
29     for (int i = 0; i < n; ++i) {
30         cin >> arr[i];
31     }
32     int q;cin >> q;
33     for (int i = 0; i < q; ++i) {
34         int l,r;cin >> l >> r;
35         l--,r--;
36         qqq[r].emplace_back(l,i);
37     }
38     for (int i = 0; i < n; ++i) {
39         add(i,arr[i]);
40         for(auto &x:qqq[i]){
41             int l = x.fi;
42             int ind = x.se;
43             ans[ind] = query(l);
44         }
45     }
46     for (int i = 0; i < q; ++i) {
47         cout << ans[i] << endl;
48     }
49 }
```

and (&) in range [l, r]

```
1 ll andRange(ll l, ll r) {
2     ll ans=0, msb = -1;
```

```

3   while(r>0) { r >>= 1; msb++;}
4   for(ll i= msb; ~i; i--) {
5       ll la = 1;
6       if((l&(la << i)) == (r&(la << i)))
7           ans += (l&(la << i));
8       else
9           break;
10  }
11  return ans;
12 }

```

Dynamic Bitset

```

1  template <int N>
2  struct Bitset {
3      using T = uint64_t;
4      static constexpr int sz_ = 64, shift = 6, AND = 63;
5      static constexpr int M = (N + AND) >> shift;
6      static constexpr T rem_ = (N % sz_ == 0) ? ~T(0) : (T(1) << (N % sz_)) - 1;
7
8      array<T, M> b{};
9
10     // ===== Basic operations =====
11     inline void set() { fill(b.begin(), b.end(), ~T(0)); b.back() &= rem_; }
12     inline void reset() { fill(b.begin(), b.end(), 0); }
13     inline void flip() { for (T &i : b) i ^= ~T(0); b.back() &= rem_; }
14
15     inline void set(int i) { b[i >> shift] |= T(1) << (i & AND); }
16     inline void reset(int i) { b[i >> shift] &= ~T(1) << (i & AND); }
17     inline void flip(int i) { b[i >> shift] ^= T(1) << (i & AND); }
18     inline bool test(int i) const { return (b[i >> shift] >> (i & AND)) & 1; }
19     inline bool operator[](int i) const { return test(i); }
20
21     // ===== Queries =====
22     inline bool any() const {
23         for (T x : b) if (x) return true;
24         return false;
25     }
26     inline bool none() const { return !any(); }
27     inline bool all() const {
28         for (int i = 0; i < M - 1; i++) if (~b[i]) return false;
29         return b.back() == rem_;
30     }
31     inline int count() const {
32         int res = 0;
33         for (T x : b) res += __builtin_popcountll(x);
34         return res;
35     }
36
37     // ===== In-place Bitwise ops (Optimized) =====
38     Bitset& operator|=(const Bitset &a) { for (int i = 0; i < M; i++)
39         b[i] |= a.b[i]; return *this; }
40     Bitset& operator&=(const Bitset &a) { for (int i = 0; i < M; i++)
41         b[i] &= a.b[i]; return *this; }
42     Bitset& operator^=(const Bitset &a) { for (int i = 0; i < M; i++)

```



```

43         b[i] ^= a.b[i]; return *this; }
44
45 Bitset operator|(const Bitset &a) const { return Bitset(*this) |= a; }
46 Bitset operator&(const Bitset &a) const { return Bitset(*this) &= a; }
47 Bitset operator^(const Bitset &a) const { return Bitset(*this) ^= a; }
48 Bitset operator~() const { Bitset res = *this; res.flip(); return res; }
49
50 bool operator==(const Bitset &a) const { return b == a.b; }
51 bool operator!=(const Bitset &a) const { return b != a.b; }
52
53 // ===== Shifts (Fixed & In-place optimized) =====
54 Bitset& operator>>=(int n) {
55     if (n >= N) { reset(); return *this; }
56     int nxt = n >> shift, sh = n & AND, sh1 = sz_ - sh;
57     for (int i = 0; i < M - nxt; i++) {
58         b[i] = b[i + nxt] >> sh;
59         if (sh && i + nxt + 1 < M) b[i] |= b[i + nxt + 1] << sh1;
60     }
61     fill(b.begin() + M - nxt, b.end(), 0);
62     return *this;
63 }
64
65 Bitset& operator<<=(int n) {
66     if (n >= N) { reset(); return *this; }
67     int nxt = n >> shift, sh = n & AND, sh1 = sz_ - sh;
68     for (int i = M - 1; i >= nxt; i--) {
69         b[i] = b[i - nxt] << sh;
70         if (sh && i - nxt - 1 >= 0) b[i] |= b[i - nxt - 1] >> sh1;
71     }
72     fill(b.begin(), b.begin() + nxt, 0);
73     b.back() &= rem_;
74     return *this;
75 }
76
77 Bitset operator>>(int n) const { return Bitset(*this) >>= n; }
78 Bitset operator<<(int n) const { return Bitset(*this) <<= n; }
79
80 // ===== Find methods =====
81 inline int find_first() const {
82     for (int i = 0; i < M; i++) if (b[i])
83         return (i << shift) + __builtin_ctzll(b[i]);
84     return -1;
85 }
86
87 inline int find_next(int pos) const {
88     if (++pos >= N) return -1;
89     int blk = pos >> shift;
90     T mask = b[blk] & (~T(0) << (pos & AND));
91     if (mask) return (blk << shift) + __builtin_ctzll(mask);
92     for (int i = blk + 1; i < M; i++) if (b[i])
93         return (i << shift) + __builtin_ctzll(b[i]);
94     return -1;
95 }
96
97 inline int find_prev(int pos) const {
98     if (--pos < 0) return -1;

```

```

99     int blk = pos >> shift;
100     T mask = b[blk] & (~T(0) >> (63 - (pos & AND)));
101     if (mask) return (blk << shift) + 63 - __builtin_clzll(mask);
102     for (int i = blk - 1; i >= 0; i--) {
103         if (b[i]) return (i << shift) + 63 - __builtin_clzll(b[i]);
104     }
105     return -1;
106 }
107 };

```

Bit Twiddle Permute

```

1  int bit_twiddle_permute(int v) { // next integer that has _pop_count(v) bits
2      int t = v | (v - 1);
3      int w = (t + 1) | (((~t & -~t) - 1) >> (__builtin_ctz(v) + 1));
4      return w;
5  }

```